
Name: Tuguinay, Nanelle

Section: 402I

Component I. Vector Indexing and Embedding

```
Start coding or generate with AI.
```

```
# ============================================================
# 1. INSTALL REQUIRED LIBRARIES
# ============================================================
!pip install -q faiss-cpu sentence-transformers transformers sentenc
```

- faiss-cpu: A library for efficient similarity search and clustering of dense vectors.
- sentence-transformers: A framework for state-of-the-art sentence, text, and image embeddings.
- transformers & sentencepiece: Libraries developed by Hugging Face for working with pre-trained NLP models.
- pandas & openpyxl: Tools for data manipulation and exporting structure data to Excel files.
- groq: The official library to interact with the Groq API for fast LLM inference.

The -q flag runs the installation in quiet mode, hiding the detailed progress logs.

```
# ============================================================
# 2. IMPORT LIBRARIES
# ============================================================
from google.colab import files
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
import pandas as pd
import pickle
import re
import os
```

```
import os
from openpyxl.styles import Font, PatternFill, Alignment

raw_text = re.sub(r'Fourteenth Congress.*?assembled::\s*', '', raw_t
```

- from google.colab import files: Used to handle file uploads from your local computer and downloads back to your machine directly within Google Colab.
- from sentence_transformers import SentenceTransformer: Imports the framework used to load pre-trained embedding models (like all-MiniLM-L6-v2) and generate vector embeddings for your text chunks.
- import faiss: Imports the Facebook AI Similarity Search library, which is used to construct a vector index and perform extremely fast similarity searches on the generated embeddings.
- import numpy as np: A standard library for working with multi-dimensional arrays, which are used to store and manipulate the raw numerical vector embeddings.
- import pandas as pd: Imports the Pandas library, used for structured data manipulation and managing data frames (like organizing chunk records).
- import pickle: Used for serializing and de-serializing Python object structures (e.g., saving or loading processed data structure states).
- import re: The regular expressions module used for parsing, cleaning, and sanitizing the raw input text.
- import os: Provides functions for interacting with the operating system, such as managing file paths and system environment variables.
- from openpyxl.styles import Font, PatternFill, Alignment: Used to apply custom styling, colors, and formatting when exporting the chunked data into an Excel sheet.

Text Preprocessing Rule

raw_text = re.sub(r'Fourteenth Congress.*?assembled::\s', '', raw_text, flags=re.DOTALL)

This line uses a regular expression (re.sub) to find any introductory text matching the pattern Fourteenth Congress...assembled:: (across multiple lines, thanks to re.DOTALL) and replaces it with an empty string. This helps sanitize the raw document text by stripping out unnecessary headers before processing.

```
# ============================================================
# 3. INDEXING PARAMETERS
# ============================================================
CHUNK_SIZE = 600
```

- CHUNK_SIZE = 600: This variable specifies the maximum number of

characters allowed per text chunk. It has been set to 600 (reduced from a larger default like 1500) because smaller, denser chunks allow the all-MiniLM-L6-v2 embedding model to produce highly focused vector representations, which improves retrieval accuracy.

```
CHUNK_OVERLAP = 150
```

- CHUNK_OVERLAP = 150 sets the number of overlapping characters between adjacent text chunks. During the document splitting process, each new chunk starts 150 characters before the previous chunk ends, ensuring that sentences and semantic contexts spanning across chunk boundaries are preserved.

```
TOP_K = 5
```

- TOP_K = 5 defines a search configuration variable. It specifies that during a vector search or document retrieval step, the system should find and return the top 5 most similar text chunks based on their embedding distance relative to the user's query.

```
EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"
```

- EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2" defines a configuration variable storing the name of the pre-trained model used for generating vector embeddings. In this pipeline, all-MiniLM-L6-v2 is selected because it is a fast, lightweight, and highly accurate sentence-transformers model that maps sentences and paragraphs into a dense 384-dimensional vector space.

```
MIN_TAIL_LENGTH = 30
```

- MIN_TAIL_LENGTH = 30 defines a safety threshold (in characters) used during the text chunking process. When dividing the document into sequential blocks, if the remaining text at the very end of a page or document is extremely short (fewer than 30 characters), the system will append that small trailing remnant to the previous chunk instead of creating an incomplete, nearly empty new chunk. This ensures that every chunk contains a meaningful amount of context.

```
# ============================================================
# 4. LOAD TEXT DATASET (VAW dataset.txt)
```

```
# 4. LOAD TEXT DATASET (VAW_dataset.txt)
# ============================================================
print("=" * 70)
print("UPLOAD DOCUMENT")
print("=" * 70)

uploaded = files.upload()
if not uploaded:
    raise RuntimeError("No file was uploaded.")
```

- Visual Header: It prints a styled header (UPLOAD DOCUMENT surrounded by lines of = characters) to clearly mark the step in your console output.
- files.upload(): This triggers a browser prompt asking you to select and upload a file (such as VAW_dataset.txt) from your local computer into Colab's temporary storage.
- Validation Check: It checks if the uploaded dictionary is empty. If you cancel the file picker without selecting a file, it raises a RuntimeError("No file was uploaded.") to prevent the rest of the script from executing with missing data.

```
txt_file = next(iter(uploaded.keys()))
with open(txt_file, "r", encoding="utf-8") as f:
    raw_text = f.read()
```

- txt_file = next(iter(uploaded.keys())): The uploaded variable is a dictionary where the keys are the names of the uploaded files. This line safely grabs the first (and in this case, only) filename from that dictionary.
- with open(txt_file, "r", encoding="utf-8") as f:: It opens the file for reading ("r") using UTF-8 encoding, which ensures that special characters and formatting are read correctly.
- raw_text = f.read(): It reads the entire text content of the file and stores it in the raw_text variable for further cleaning and preprocessing.

```
# Normalize whitespace
cleaned_text = re.sub(r"\s+", " ", raw_text).strip()
pages = [{"page": 1, "text": cleaned_text}]  # Treating txt content
```

- cleaned_text = re.sub(r"\s+", " ", raw_text).strip(): This line uses a regular expression to find all sequences of whitespace characters (like multiple spaces, tabs, and newlines) and consolidates them into a single space (" "). .strip() removes any leading or trailing spaces. This standardizes the formatting so that line breaks do not interfere with text chunking downstream.

- pages = [{"page": 1, "text": cleaned_text}]: Since plain text files (.txt) do not naturally have structural page breaks (unlike PDFs), this line packages the entire cleaned text as "Page 1" within a dictionary. Putting it inside a list like this keeps the data structure consistent with multi-page pipelines, allowing you to use the exact same create_chunks function designed for page-aware documents.

```
# ============================================================
# 5. CREATE CHUNKS
# ============================================================
def create_chunks(pages, chunk_size, chunk_overlap, min_tail_length
    chunks = []
    chunk_id = 0
    step_size = chunk_size - chunk_overlap
```

- def create_chunks(pages, chunk_size, chunk_overlap, min_tail_length=50):: This defines the function create_chunks which takes the list of pages, your desired chunk size, the overlap margin, and an optional minimum tail length threshold (defaulting to 50 if not specified)
- chunks = []: Initializes an empty list that will store the final dictionary objects representing each individual text chunk along with its metadata (such as chunk ID and page number).
- chunk_id = 0: Sets up a counter starting at 0 to assign a unique, incremental identifier to every generated chunk.
- step_size = chunk_size - chunk_overlap: Computes the stride or step size for the sliding window algorithm. For example, if your chunk_size is 600 characters and your chunk_overlap is 150 characters, the window moves forward by 450 characters (step_size) on each iteration. This guarantees that 150 characters of overlap are preserved between successive chunks.

```
    for page_data in pages:
        page_number = page_data["page"]
        page_text = page_data["text"]
        start = 0
        page_chunk_positions = []
```

- for page_data in pages:: This initiates a loop that iterates over each page (represented as a dictionary) in the pages list.
- page_number = page_data["page"]: It extracts the current page number (e.g., 1) so that any chunk generated from this text can be tagged with its corresponding source page for metadata tracking.
- page_text = page_data["text"]: It retrieves the raw, cleaned string content belonging to that specific page to prepare it for splitting.

belonging to that specific page to prepare it for splitting.

- start = 0: Resets the character pointer to the beginning of the page (index 0) for the sliding window algorithm.
- page_chunk_positions = []: Initializes an empty list to keep track of the indices of the generated chunks on the current page. This tracking is crucial for the safety check later, which allows appending small trailing text (tails) to the last valid chunk on the page.

```
        while start < len(page_text):
            remaining = len(page_text) - start
```

- while start < len(page_text):: This is the main control loop for chunking the current page. It continues executing as long as the character pointer (start) has not reached the end of the page's text.
- remaining = len(page_text) - start: This line calculates how many characters of text are left to process on the current page. Tracking this value is essential for the next step, where we check if the remaining text is too short (a 'tail') to form a useful standalone chunk.

```
                if remaining < min_tail_length and page_chunk_positions
                    previous_position = page_chunk_positions[-1]
                    previous_chunk = chunks[previous_position]
                    previous_start = previous_chunk["_start"]
                    previous_chunk["text"] = page_text[previous_start:]
                    break
```

- if remaining < min_tail_length and page_chunk_positions:: This checks if the remaining characters left on the page (remaining) are fewer than the safety threshold (min_tail_length, which is 30 in your configuration). It also ensures that at least one chunk has already been successfully created on this page (page_chunk_positions is not empty), so there is a previous chunk available to absorb the tail.
- previous_position = page_chunk_positions[-1]: It retrieves the list index of the very last chunk that was processed on this page.
- previous_chunk = chunks[previous_position]: It accesses the dictionary representing that last chunk from the global chunks list.
- previous_start = previous_chunk["_start"]: It looks up the character index where that last chunk originally started.
- previous_chunk["text"] = page_text[previous_start:].strip(): Instead of creating a new, tiny chunk, this expands the previous chunk to include everything from its original start point all the way to the very end of the page (page_text[previous_start:]). It then trims off any surrounding spaces

(page_text[previous_start:]). It then trims off any surrounding spaces.

- break: Since the entire remaining text of the page has now been merged into the previous chunk, the loop exits early, ending the chunking process for this page.

```
end = min(start + chunk_size, len(page_text))
```

- start + chunk_size: This calculates where the chunk would end if it used the full requested chunk size (600 characters in your settings).
- len(page_text): This is the total number of characters on the current page.
- min(...): The min function ensures that if the remaining text on the page is shorter than the configured chunk_size, the chunk boundary stops exactly at the end of the text (len(page_text)) rather than trying to read past the end of the string.

```
chunk_text = page_text[start:end].strip()
```

- page_text[start:end]: This uses Python string slicing to extract the sequence of characters starting at the start index up to (but not including) the end index calculated in the previous step.
- .strip(): This string method removes any leading or trailing whitespace (such as empty spaces, tabs, or newlines) from the sliced segment. This ensures that your chunks don't start or end with unnecessary blank space, keeping the text neat and optimally formatted for generating clean vector embeddings.

```
if chunk_text:
    chunks.append({
        "chunk_id": chunk_id,
        "page": page_number,
        "text": chunk_text,
        "_start": start
    })
```

- if chunk_text:: This conditional check ensures that the extracted string is not empty or just whitespace. If the string is empty, the code skips creating a chunk to prevent storing blank elements in your database.
- chunks.append({ ... }): If valid, it appends a new dictionary representing this chunk to your main chunks list.
- "chunk_id": chunk_id: Assigns the current incremental ID to this chunk for unique tracking.
- "page": page_number: Links the chunk to its original page number so you can

- "page": page_number: Links the chunk to its original page number so you can track where the information came from.
- "text": chunk_text: Stores the actual text content that will be vectorized and indexed.
- "_start": start: Temporarily stores the starting character index of this chunk. This is a helper key used to trace where the chunk began on the page in case a final small tail of text needs to be appended to it later.

```
                page_chunk_positions.append(len(chunks) - 1)
```

- len(chunks) - 1: Since Python lists are zero-indexed, len(chunks) - 1 calculates the exact index of the chunk we just appended to the chunks list.
- page_chunk_positions.append(...): This appends that index to the page-specific list page_chunk_positions.

```
                chunk_id += 1
```

- chunk_id += 1: After a text chunk is successfully created and appended to the chunk list, this line increments the chunk_id counter by 1. This ensures that the next text chunk generated during the sliding-window process receives a fresh, unique, and sequential numeric ID (e.g., moving from ID 0 to ID 1). This ID is essential for mapping, sorting, and referencing specific retrieved passages during semantic search.

```
            start += step_size
```

- start += step_size: This increments the start character pointer by the calculated step_size (which is chunk_size - chunk_overlap, or 450 characters in your settings). By moving the pointer forward by 450 characters instead of the full 600, the system ensures that the next chunk starts exactly 150 characters before the current one ends. This creates the configured overlap that preserves the continuous semantic context between adjacent text blocks.

```
    for chunk in chunks:
        chunk.pop("_start", None)
    return chunks
```

- for chunk in chunks:: This loops through every generated chunk dictionary inside your main list.
- chunk.pop("_start", None): This removes the key _start from each chunk

- chunk.pop("_start", None): This removes the key _start from each chunk dictionary. Since _start was only a temporary pointer used to locate and attach 'tails' (short leftover segments) to preceding chunks, it is no longer needed after the main loop finishes. Removing it keeps the chunk data clean, lightweight, and structured strictly with relevant production keys (chunk_id, page, and text). The None default argument ensures that if a chunk somehow doesn't have a _start key, the code runs smoothly without throwing a KeyError.
- return chunks: Finally, this exits the function and hands back the complete, finalized list of structured text chunks, ready to be exported to Excel or converted into vector embeddings.

```
chunks = create_chunks(pages, CHUNK_SIZE, CHUNK_OVERLAP, MIN_TAIL_L
```

- create_chunks(...): It calls the chunking function we just defined and analyzed. pages: It passes the list containing your cleaned document text (packaged as Page 1).
- CHUNK_SIZE: It passes the maximum chunk length parameter (600 characters).
- CHUNK_OVERLAP: It passes the sliding window overlap parameter (150 characters) to maintain context continuity.
- MIN_TAIL_LENGTH: It passes the safety threshold parameter (30 characters) to prevent tiny, isolated trailing fragments.
- chunks =: The finalized list of formatted, non-empty chunk dictionaries returned by the function is stored in the chunks variable, ready to be exported to Excel and ingested into your FAISS vector database.

```
if not chunks:
    raise RuntimeError("No chunks were generated.")
```

- if not chunks:: This checks if the chunks list is completely empty. In Python, an empty list evaluates to False in a boolean context, so not chunks becomes True if no chunks were created.
- raise RuntimeError("No chunks were generated."): If the list is empty, this halts the entire notebook execution immediately and throws a runtime error with a descriptive message. This prevents downstream steps (such as exporting to Excel, generating vector embeddings, or training/indexing a FAISS database) from running on non-existent data, which would otherwise crash with cryptic errors.

```
```

```
# ============================================================
# 6. EXPORT CHUNKS TO EXCEL
# ============================================================
chunk_records = []
```

- chunk_records = []: This initializes an empty Python list. It will act as a temporary accumulator where we will store the restructured information of each text chunk (including its ID, source filename, page number, length, and content) as individual dictionaries. Once populated, this list will be converted into a structured Pandas DataFrame to easily generate and save your final .xlsx file.

```
for chunk in chunks:
    chunk_records.append({
        "Chunk_ID": chunk["chunk_id"],
        "Source_Document": txt_file,
        "Page_Number": chunk["page"],
        "Character_Count": len(chunk["text"]),
        "Chunk_Size_Setting": CHUNK_SIZE,
        "Chunk_Overlap_Setting": CHUNK_OVERLAP,
        "Chunk_Text": chunk["text"]
    })
```

- for chunk in chunks: This initiates a loop that iterates over each chunk dictionary in your chunks list, allowing us to process and format them one by one.
- chunk_records.append({ ... }) For every iteration, a new dictionary containing highly structured metadata and text is appended to the chunk_records list. This acts as a database-like row structure.
- "Chunk_ID": chunk["chunk_id"] Maps the unique, sequential numeric identifier of the chunk to the key Chunk_ID.
- "Source_Document": txt_file Stores the name of the source text file (e.g., VAW_dataset.txt) to keep track of which document the chunk originated from.
- "Page_Number": chunk["page"] Logs the corresponding page number (or unified source block) from which the text segment was extracted.
- "Character_Count": len(chunk["text"]) Calculates and saves the exact length (in characters) of the finalized chunk string, useful for validating your splitting constraints.
- "Chunk_Size_Setting": CHUNK_SIZE & "Chunk_Overlap_Setting": CHUNK_OVERLAP Record your static sliding-window parameters (600 and 150 respectively) as columns. This ensures that every exported row carries its configuration metadata for easier analysis and debugging later.
- "Chunk_Text": chunk["text"] Stores the actual cleaned, sliced textual content

of the chunk.

```
chunks_df = pd.DataFrame(chunk_records)
base_name = os.path.splitext(os.path.basename(txt_file))[0]
EXCEL_FILE = f"{base_name}_chunks_inspection.xlsx"
chunks_df.to_excel(EXCEL_FILE, index=False)
files.download(EXCEL_FILE)
```

- chunks_df = pd.DataFrame(chunk_records) This converts your accumulated list of structured dictionaries (chunk_records) into a Pandas DataFrame. Each dictionary key (like Chunk_ID or Chunk_Text) becomes a column, and each dictionary becomes a row.
- base_name = os.path.splitext(os.path.basename(txt_file))[0] This strips away the folder path and the file extension from your source text file. For example, if txt_file is "VAW_dataset (5).txt", it extracts just "VAW_dataset (5)". This keeps the output filename dynamically linked to your original input.
- EXCEL_FILE = f"{base_name}_chunks_inspection.xlsx" Creates the target export filename by appending _chunks_inspection.xlsx to the base name extracted above.
- chunks_df.to_excel(EXCEL_FILE, index=False) Exports the Pandas DataFrame directly into an Excel worksheet. Specifying index=False prevents Pandas from writing the auto-generated row numbers as an extra unnamed column in your Excel sheet.
- files.download(EXCEL_FILE) Uses Google Colab's native file utility to trigger an automatic browser download, sending the compiled Excel file straight to your local downloads folder.

```
# ============================================================
# 7. CREATE FAISS INDEX & EMBEDDINGS
# ============================================================
embedding_model = SentenceTransformer(EMBEDDING_MODEL_NAME)
```

- embedding_model = SentenceTransformer(EMBEDDING_MODEL_NAME) This line initializes and loads your text embedding model. It instantiates the SentenceTransformer class using the model name stored in your configuration variable (EMBEDDING_MODEL_NAME, which is set to 'all-MiniLM-L6-v2'). When this line runs, it loads the model weights into memory (downloading them from Hugging Face if they are not already cached locally). This model will be used to translate your text chunks into dense 384-dimensional vector representations that capture their semantic meaning.

```
chunk_texts = [chunk["text"] for chunk in chunks]
```

```
chunk_texts = [chunk["text"] for chunk in chunks]
```

- [chunk["text"] for chunk in chunks] This is a standard Python list comprehension. It acts as a lightweight, highly efficient shorthand for a loop that extracts a specific element from a list of dictionaries.
- for chunk in chunks It iterates through your main list of chunks (where each element is a dictionary containing metadata like chunk_id, page, and text).
- chunk["text"] During each iteration, it accesses and extracts the value associated with the "text" key—which is the actual cleaned text string of that chunk.
- chunk_texts = This compiles all those extracted text strings into a new, single flat list of strings called chunk_texts. This purified list is precisely what the SentenceTransformer model expects as input to generate your vector embeddings in the next step, without any of the accompanying dictionary metadata getting in the way.

```
embeddings = embedding_model.encode(
    chunk_texts,
    convert_to_numpy=True,
    normalize_embeddings=True,
    show_progress_bar=True
).astype("float32")
```

- embedding_model.encode(...) This calls the encoding engine of your loaded SentenceTransformer model to process the list of raw text segments (chunk_texts). It passes each chunk through its neural network to calculate a dense vector representation of its semantic meaning.
- convert_to_numpy=True Instructs the model to output the final embeddings directly as a standard NumPy multi-dimensional array, making it immediately compatible with numerical libraries like FAISS.
- normalize_embeddings=True Normalizes each generated vector to have a unit length of 1 (an L2 norm of 1.0). Normalizing your embeddings upfront is a best practice; it ensures that calculating the Inner Product (cosine similarity) during search is mathematically equivalent to calculating standard Cosine Similarity, which makes retrieval faster and more reliable.
- show_progress_bar=True Displays an interactive, real-time loading bar in the notebook execution output so you can monitor the progress of the vectorization, which is helpful when processing large datasets.
- .astype("float32") Casts the resulting NumPy array data type to 32-bit floating point precision. This step is critical because FAISS indices strictly require input vectors to be in float32 format to construct the index and perform similarity search calculations.

perform similarity search calculations.

```
embedding_dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(embedding_dimension)
index.add(embeddings)
```

- embedding_dimension = embeddings.shape[1] This retrieves the size or dimensionality of your generated vector embeddings from the NumPy array (embeddings). Since you are using the all-MiniLM-L6-v2 model, this value will be exactly 384 (meaning each text chunk is represented by a list of 384 numbers). FAISS needs this dimension size to build the index structure properly.
- index = faiss.IndexFlatIP(embedding_dimension) This instantiates a FAISS Index structure. Specifically, IndexFlatIP stands for Flat Inner Product. Because we pre-normalized your embeddings to unit length in the previous step, calculating the inner product during a query search becomes mathematically identical to calculating Cosine Similarity. The term "Flat" means that the index stores raw vectors without compressing or clustering them, ensuring perfect (100% accurate) nearest-neighbor search results.
- index.add(embeddings) This feeds your generated float32 embeddings array into the FAISS index. Once added, the index is fully populated, memory-mapped, and ready to perform lightning-fast similarity search lookups.

```
chunk_lookup = {i: chunk for i, chunk in enumerate(chunks)}
```

- {i: chunk for i, chunk in enumerate(chunks)} This is a standard Python dictionary comprehension. It is a clean, optimized way to construct a dictionary on the fly by looping over an iterable sequence.
- enumerate(chunks) This function iterates through your chunks list while keeping track of the current index. For each item in the list, it yields a pair: i (the index integer, starting from 0) and chunk (the actual chunk dictionary object containing metadata and text).
- i: chunk This defines the key-value structure of the resulting dictionary. The index i (e.g., 0, 1, 2, ...) serves as the key, and the corresponding chunk dictionary is stored as its value.
- chunk_lookup = This stores the newly built map in a variable called chunk_lookup. This dictionary allows for O(1) constant-time lookup. During vector search, FAISS returns the index of the most similar embedding vector. Using chunk_lookup[idx], the chatbot can instantly retrieve the matching source document chunk and metadata without needing to search or scan through the entire list.

through the entire list.

```
def retrieve_documents(query, top_k=TOP_K):
    if not query or not query.strip():
        raise ValueError("Query cannot be empty.")
```

- def retrieve_documents(query, top_k=TOP_K): This defines your primary document retrieval function. It accepts a search query (as a string) and an optional parameter top_k (the number of matches to return). If the caller does not specify top_k, it defaults to the value stored in your configuration variable TOP_K (which is 5).
- if not query or not query.strip(): This is a safety guardrail. It checks two conditions: first, if the query object is None or completely empty (not query), and second, if the query contains only blank spaces (not query.strip()).
- raise ValueError("Query cannot be empty.") If either validation condition is met (meaning the search query is empty or just spaces), the function immediately halts execution and raises a ValueError. This prevents the pipeline from attempting to generate vector embeddings for a blank string, which would result in meaningless similarity scores and index errors.

```
    q_emb = embedding_model.encode([query], convert_to_numpy=True,
    scores, indices = index.search(q_emb, min(top_k, index.ntotal))
```

- q_emb = embedding_model.encode([query], ...) This converts your textual search query into a vector embedding using the same all-MiniLM-L6-v2 model. The query is wrapped in a list [query] because the encoder expects a batch input. Just like the document chunks, the resulting query vector is normalized (normalize_embeddings=True), converted to a NumPy array (convert_to_numpy=True), and cast to float32 precision (.astype("float32")).
- index.search(q_emb, min(top_k, index.ntotal)) This runs the core search on your populated FAISS index. It compares your query vector (q_emb) against all the stored document chunk vectors using the Flat Inner Product metric (which acts as cosine similarity because both vectors are pre-normalized).
- min(top_k, index.ntotal) This is a safety check. It ensures that if the user requests more results (top_k) than there are total chunks stored in the index (index.ntotal), the search won't crash or ask for non-existent records. Instead, it limits the search to the maximum available number of indexed chunks.
- scores, indices = This captures the search results into two arrays:
    - scores: An array containing the calculated similarity scores (cosine similarity values) for the closest matches.

- indices: An array containing the integer positions/IDs of the closest matches, which map directly to our chunk_lookup dictionary.

```
        results = []
        for rank, idx in enumerate(indices[0]):
            if idx == -1:
                continue
            chunk = chunk_lookup[int(idx)]
            results.append({
                "rank": rank + 1,
                "chunk_id": chunk["chunk_id"],
                "page": chunk["page"],
                "score": float(scores[0][rank]),
                "text": chunk["text"]
            })
        return results

    print("Indexing Complete. Vector store is ready.")
```

- results = [] Initializes an empty list to collect the formatted search results that will be returned to the caller.
- for rank, idx in enumerate(indices[0]): Iterates through the nearest neighbor indices returned by FAISS. enumerate provides a zero-based rank count (e.g., 0, 1, 2) alongside the corresponding index idx of the matched vector in the FAISS database.
- if idx == -1:

- continue This is a safety check. If FAISS cannot find enough valid nearest neighbors to satisfy your requested top_k, it fills the remaining slots with -1. This condition simply skips those empty placeholders.
- chunk = chunk_lookup[int(idx)] Uses the fast O(1) dictionary lookup we created earlier to convert the index integer (idx) back into the original, full text chunk dictionary containing its page number, text, and original chunk ID.
- results.append({ ... }) Appends a clean, dictionary-based record for each matched chunk to the results list. It registers:
    - "rank": The 1-based similarity rank (rank + 1), starting at 1 for the closest match.
    - "chunk_id": The original chunk ID.
    - "page": The page number of the source document.
    - "score": The cosine similarity score computed by FAISS, explicitly cast to a standard Python float.
    - "text": The raw text content of the chunk.
- return results Exits the retrieval function and returns your ranked list of matching chunks to the RAG pipeline.
- print("Indexing Complete. Vector store is ready.") Prints a success message to confirm that the vector index building and retrieval helper setup have successfully executed.

Component II: Prompt and Retrieval

```
# ============================================================
# 1. INITIALIZE GROQ LLM CLIENT & SECRETS
# ============================================================
import os
from groq import Groq
from google.colab import userdata

os.environ["GROQ_API_KEY"] = userdata.get('GROQ_API_KEY')
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

LLM_NAME = "openai/gpt-oss-20b"
RAG_TOP_K = 3
FALLBACK_ANSWER = "I cannot find the answer in the provided documen

print("=" * 70)
print("MAGNA CARTA OF WOMEN RAG CHATBOT (GROQ API)")
print("=" * 70)
print("Using Model:", LLM_NAME)
```

- Library Imports (import os, from groq import Groq, from google.colab import userdata): Loads the standard operating system module to manage environment configurations, the official Groq client library to enable fast inference, and Google Colab's native credentials helper to safely access keys.
- API Key Setup (os.environ["GROQ_API_KEY"] = userdata.get('GROQ_API_KEY')): Securely fetches your secret API key stored in Colab's 'Keys' tab (under the name GROQ_API_KEY) and saves it directly to the system environment variables so that the Groq SDK can authenticate automatically.
- Client Initialization (client = Groq(...)): Instantiates the Groq API connection client using the retrieved credentials, enabling the chatbot to send context-aware prompts and receive responses.
- RAG Parameters Definition (LLM_NAME, RAG_TOP_K, FALLBACK_ANSWER): Sets up the core configuration: it assigns the target model identifier (openai/gpt-oss-20b), configures the chatbot to retrieve only the top 3 most relevant context chunks to fit the model's window, and defines a strict fallback error string to display when no matching context is located.
- Diagnostics Printout (print(...)): Displays a structured header in the console to confirm that the RAG pipeline has successfully initialized and shows which language model is currently active.

```
# ============================================================
# 2. PROMPT BUILDER WITH GUARDRAILS
# ============================================================
def create_rag_prompt(question, retrieved_documents):
    context_blocks = []
    for result in retrieved_documents:
        page = result.get("page", "Unknown")
        chunk_id = result.get("chunk_id", "Unknown")
        text = result.get("text", "").strip()
        context_blocks.append(f"[Section / Page {page}; Chunk {chun
```

- Function Definition (def create_rag_prompt(question, retrieved_documents):): This defines a prompt-building helper function. It accepts the user's raw string question along with a list of dictionaries (retrieved_documents) representing the most relevant context blocks retrieved from our FAISS vector store.
- List Initialization (context_blocks = []): Initializes an empty Python list designed to collect formatted, human-readable text blocks containing both the raw text of the document passages and their tracking metadata (like page and chunk ID).
- Document Iteration (for result in retrieved_documents:): Loops over each

individual retrieved document dictionary returned by the similarity search.

- Safe Metadata Retrieval (result.get(...)): Uses Python's safe .get() dictionary method to fetch the page, chunk_id, and text attributes. Using .get("key", "default") acts as a guardrail, ensuring that if any metadata field is missing, it falls back to a default value (like "Unknown" or an empty string "") instead of crashing the pipeline with a KeyError.
- Formatting and Appending (context_blocks.append(f"...")): For each matched segment, it cleans up leading and trailing whitespaces using .strip() and structures a highly descriptive context block using an f-string: [Section / Page {page}; Chunk {chunk_id}]\n{text}. This explicitly tells the downstream language model exactly where each piece of information originated from, enabling precise referencing.

```
        context_str = "\n\n".join(context_blocks)
```

- context_blocks: This is a list containing the formatted text chunks that the retriever found (e.g., each item includes the source section/page label and the chunk text itself).
- "\n\n": This is the separator string. It represents two newline characters (a double line break or a blank line between each block).
- .join(...): This is a built-in Python string method that stitches all the separate elements in the context_blocks list together sequentially, inserting the "\n\n" separator in between each block.

```
        system_instruction = (
            "You are a specialized legal information assistant for Repu
            "(The Magna Carta of Women). Answer the user's question cle
            "the provided document context. If the answer is present in
            "If the evidence is insufficient, state exactly: "
            "'I cannot find the answer in the provided document.'"
        )
```

This code snippet defines your System Instruction (or System Prompt). It sets the core rules, persona, and behavioral boundaries for the Large Language Model (openai/gpt-oss-20b) before it reads any user questions or document chunks.

- user_prompt = f"...": Dynamic Formatting: Uses a Python f-string (formatted string literal) to dynamically stitch together variables into a single text prompt.
- Question: {question}: User Query: Inserts the actual question typed by the user (e.g., "What is the official short title of Republic Act No. 9710?") right

into the prompt structure so the model knows what it is supposed to answer.

- Retrieved Document Passages:\n{context_str}: Injected Context: Inserts the combined context string (context_str) that you created in the previous line—containing the top matching legal text chunks from your FAISS database. This gives the model the exact source material it is allowed to use.
- \n\nAnswer: Generation Cue: Acts as a prompt engineering cue. It tells the LLM where the reference data ends and where it is expected to output its generated response.
- return system_instruction, user_prompt: Returning the Tuple: Sends both the behavior rules (system_instruction) and the formatted input (user_prompt) back to whatever function called it, so they can be packaged together into the API request sent to Groq.

```
# ============================================================
# 3. GENERATION FUNCTION
# ============================================================
def generate_answer(system_instruction, user_prompt):
    try:
        completion = client.chat.completions.create(
            model=LLM_NAME,
            messages=[
                {"role": "system", "content": system_instruction},
                {"role": "user", "content": user_prompt}
            ],
            temperature=0.1,
            max_tokens=400
        )
        return completion.choices[0].message.content.strip()
    except Exception as error:
        return f"Error communicating with Groq API: {str(error)}"
```

- def generate_answer(system_instruction, user_prompt): Function Signature: Defines a Python function that takes two arguments: the behavioral rules (system_instruction) and the input context/question (user_prompt).
- try: Error Boundary: Starts a try-except block to safely catch any connection drops, authentication failures, or API rate limits without crashing your entire notebook.
- completion = client.chat.completions.create(...)
    - API Call: Sends a request to the Groq client to generate text completion.
    - model=LLM_NAME: Specifies which model to use (in your case, openai/gpt-oss-20b).
    - messages=[...]: Passes the conversation roles structure required by chat completion APIs, separating the rules (system) from the data (user).

    - temperature=0.1: Sets the randomness level very low (close to 0). This makes the model's output highly deterministic, precise, and faithful to your text chunks, which is ideal for legal document retrieval.
    - max_tokens=400: Caps the maximum length of the generated response to keep answers concise and focused.
- return completion.choices[0].message.content.strip(): Extraction: Navigates the JSON response returned by Groq, extracts the text content of the model's answer, and strips away any extra whitespace.
- except Exception as error: Exception Handling: If anything goes wrong during the API request, it catches the error and returns a clean, readable error string instead of breaking your program.

```
# ============================================================
# 4. FULL RAG QUERY PIPELINE
# ============================================================
def rag_query(question, top_k=RAG_TOP_K):
    question = question.strip()
    if not question:
        return "Please enter a question.", []

    retrieved_documents = retrieve_documents(question, top_k=top_k)
    if not retrieved_documents:
        return FALLBACK_ANSWER, []

    sys_instr, user_prm = create_rag_prompt(question, retrieved_doc
    answer = generate_answer(sys_instr, user_prm)
    return answer, retrieved_documents
```

- def rag_query(question, top_k=RAG_TOP_K): Function Signature: Defines the main pipeline function. It accepts a user query string and an optional top_k parameter (which defaults to your configured value of 3 or 5).
- question = question.strip(): Input Cleaning: Strips any leading or trailing whitespace from the user's input question.
- if not question: / return "Please enter a question.", []: Validation Guardrail: Checks if the user submitted an empty string or blank spaces. If they did, it immediately halts and returns a polite reminder message instead of running an empty search.
- retrieved_documents = retrieve_documents(question, top_k=top_k): Retrieval Phase: Calls your FAISS vector search function (retrieve_documents) to embed the user's question and pull the top matching document chunks from your vector database.
- if not retrieved_documents: / return FALLBACK_ANSWER, []: Fallback Guardrail: If the vector search returns nothing, it immediately returns your

predefined fallback phrase ("I cannot find the answer in the provided document.").

- sys_instr, user_prm = create_rag_prompt(question, retrieved_documents): Prompt Construction: Passes the user's question and the retrieved chunks into your prompt builder function to format the system instructions and the context-injected user prompt.
- answer = generate_answer(sys_instr, user_prm): Generation Phase: Sends the finalized prompt and system rules to the Groq API (openai/gpt-oss-20b) to generate the final text response.
- return answer, retrieved_documents: Output Return: Returns both the generated text answer and the list of source chunks back to the user or interface loop so you can view the response provenance alongside the text.

```
# ============================================================
# 5. CONTINUOUS CHATBOT LOOP (Type 'stop' or 'exit' to quit)
# ============================================================
print("\n" + "=" * 70)
print("Chatbot is ready! Type your questions below.")
print("Type 'stop' or 'exit' when you want to end the session.")
print("=" * 70 + "\n")

while True:
    user_question = input("Please enter a question: ").strip()

    # Check if user wants to exit
    if user_question.lower() in ["stop", "exit", "quit"]:
        print("\nSession ended. Good luck with your lab submission!
        break

    if not user_question:
        print("Please enter a valid question.\n")
        continue

    answer, sources = rag_query(user_question)

    print(f"\nGenerated Answer:\n{answer}")
    print("\n" + "-" * 50)
    print("Retrieved Source Chunks:")
    for idx, doc in enumerate(sources):
        print(f"[Chunk {idx+1} | ID: {doc.get('chunk_id')} | Page:
    print("-" * 50 + "\n")
```

- print(...) and header messages: Session Banner: Prints a clear visual banner in the Colab output so you know the chatbot is initialized and ready to receive your test questions.
- while True: Infinite Loop: Starts a continuous loop that keeps the chatbot running indefinitely, repeatedly asking you for input until you explicitly tell it to

running indefinitely, repeatedly asking you for input until you explicitly tell it to stop.

- user_question = input("Please enter a question: ").strip(): Interactive Input: Displays an input text box in Google Colab where you can type your questions (like your Easy, Average, and Difficult test cases), automatically stripping any accidental leading or trailing spaces.
- if user_question.lower() in ["stop", "exit", "quit"]: / break: Exit Guardrail: Checks your input (case-insensitively). If you type "stop", "exit", or "quit", it prints a closing message and breaks out of the while loop to end the session gracefully.
- if not user_question: / continue: Empty Input Check: If you press Enter without typing anything, it prints a prompt reminding you to enter a valid question and skips back to the top of the loop (continue).
- answer, sources = rag_query(user_question): Pipeline Execution: Calls your core RAG function, passing your question through vector retrieval, prompt building, and the Groq LLM API to get the final answer and source chunks.
- print(f"\nGenerated Answer:\n{answer}"): Answer Display: Prints the AI-generated response clearly underneath your question.
- for idx, doc in enumerate(sources): Source Provenance Loop: Iterates through the retrieved source chunks used to generate the answer, printing out their specific Chunk ID and Page number so you can verify where the legal information came from.
